S3-BUCKET TASK

Create a Jira ticket for this assignment, work on the tasks listed below, and submit the task completion document by EOD with clear steps and required evidence (outputs/screenshots)

Create an S3 bucket and upload some objects to S3.

Deploy a static website in the S3 bucket.

Enable cross-region replication on S3 buckets.

Configure a bucket policy so only the Admin user can see the objects of the S3 bucket.

Set up lifecycle policies to automatically transition or delete objects based on specific criteria.

Push some objects to S3 using the AWS CLI.

Write a Bash script to create an S3 bucket.

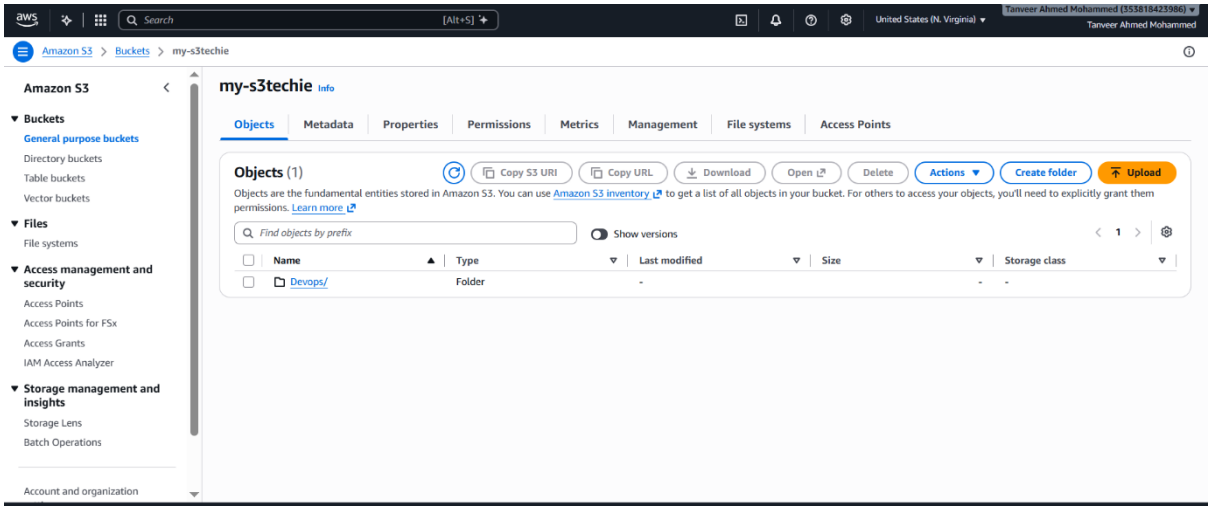
Upload a 1 GB file to S3 using the CLI.

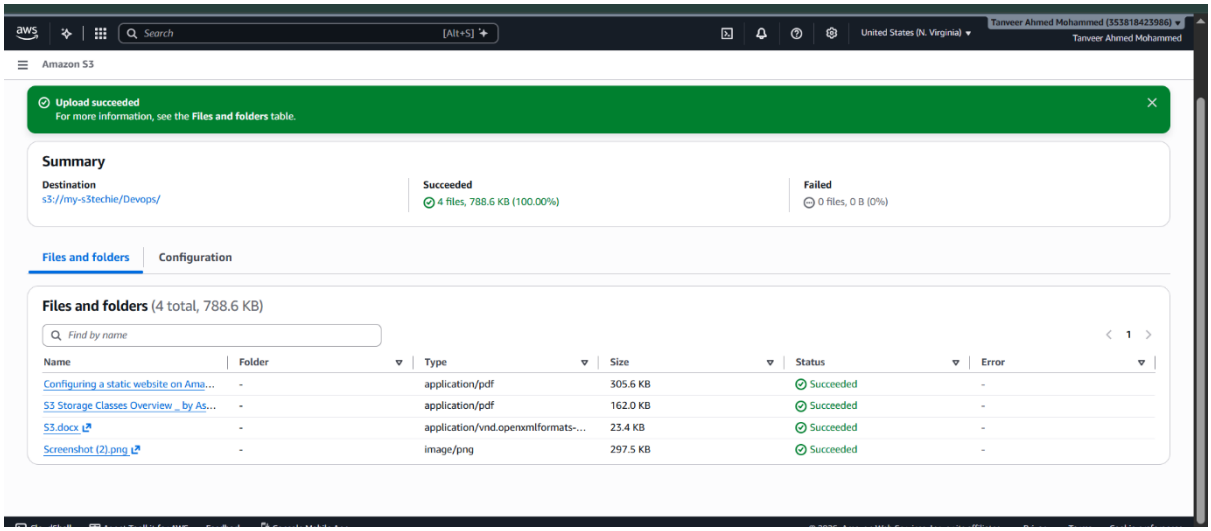
1) Create an S3 bucket and upload some objects to S3.

Task Execution Guide & Implementation Steps

1. Create an S3 Bucket and Upload Objects

- **Console Steps:**
    1. Open the **Amazon S3 Console** and click **Create bucket**.
    2. Enter a globally unique bucket name (e.g., company-app-assets-2026).
    3. Select your preferred Region (e.g., us-east-1).
    4. Keep "Block Public Access" enabled unless required for static hosting, then click **Create bucket**.
    5. Click into the bucket, click **Upload**, add sample files (e.g., index.html, logo.png), and click **Upload**.

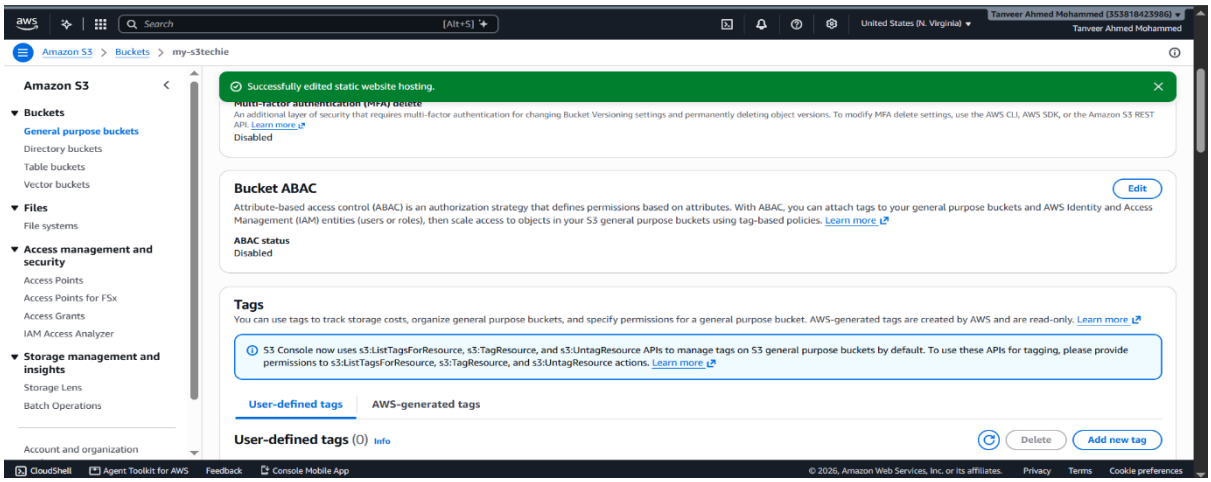


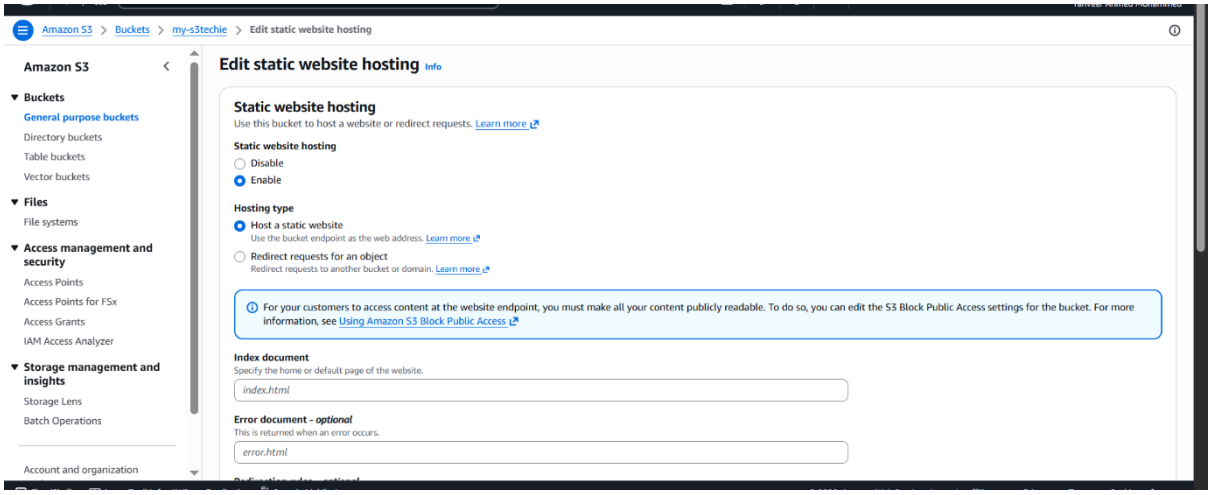


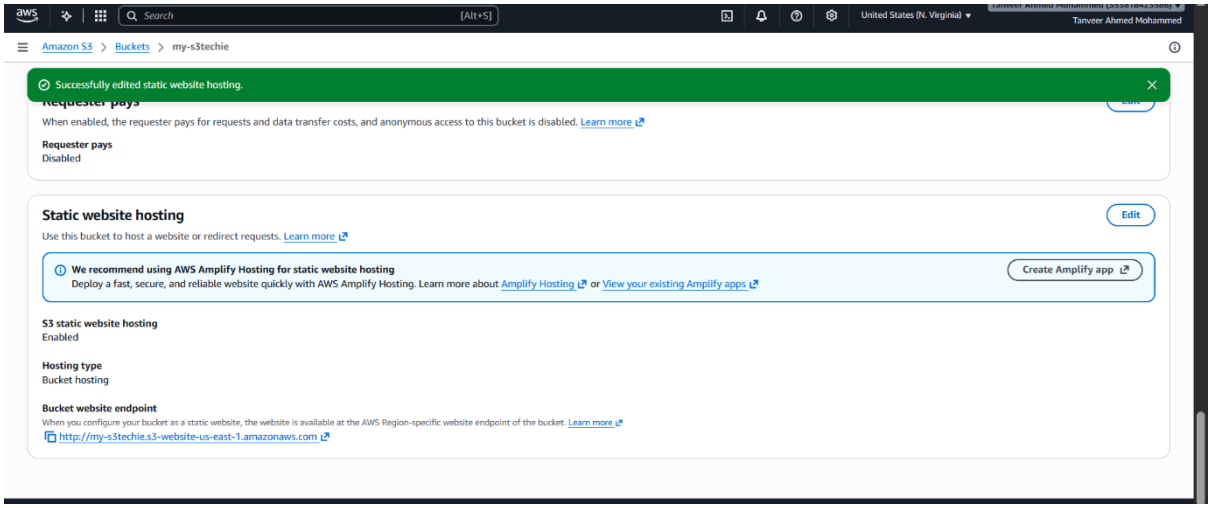
2) **Deploy a static website in the S3 bucket.**

Go to your S3 bucket settings and uncheck **Block all public access** (acknowledge the warning).

- Go to the **Properties** tab, scroll to **Static website hosting**, and click **Enable**.
- Set the Index document to index.html and click **Save changes**.









Hello World

---

3) Enable cross-region replication on S3 buckets.

Cross-region replication automatically and asynchronously copies object updates across buckets in different AWS regions. To set this up properly, you need a **Source bucket** (e.g., in us-east-1) and a

Destination bucket in a different region (e.g., us-west-2), and **Bucket Versioning** must be enabled on both.

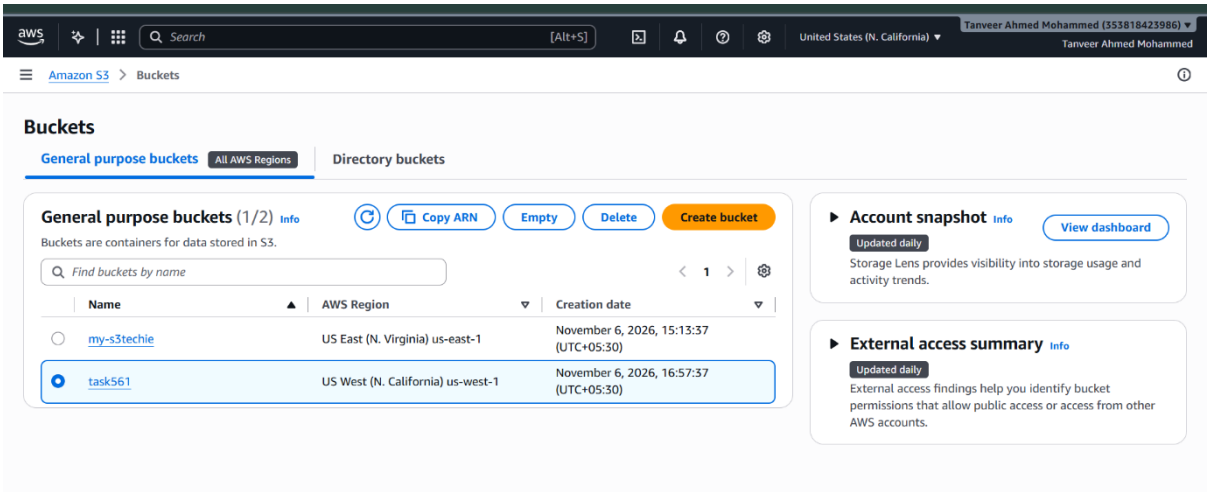
1.Enable Bucket Versioning: Required for both source and destination.

1. Open the Amazon S3 console and click on your source bucket (my-s3techie).
2. Go to the **Properties** tab.
3. Under **Bucket Versioning**, click **Edit** and select **Enable**, then save.
4. Repeat this exact step for your **Destination bucket** in the secondary region.

Verification: The Bucket Versioning card for both buckets should display "Enabled".

2.Create and Prepare Destination Bucket: Create a second bucket in another region.

1. Click **Create bucket** in the S3 console.
2. Give it a globally unique name (e.g., my-s3techie-replica-west).
3. Select a **different AWS Region** from your source bucket USA California.
4. Ensure versioning is enabled on this bucket as well.

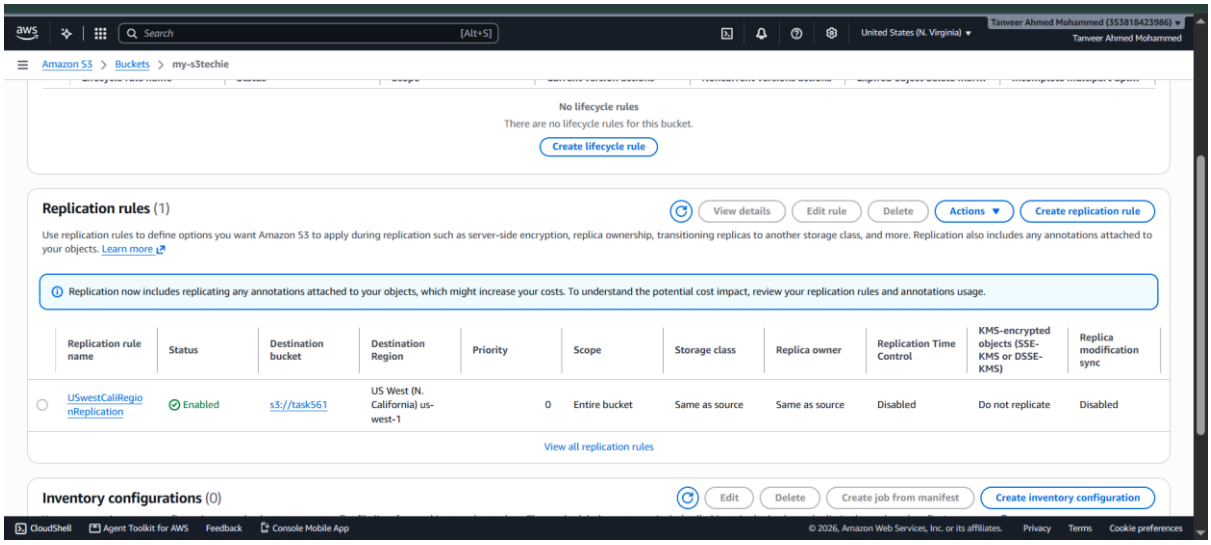


Verification: The new bucket appears in your S3 bucket list with the target region specified.

3.Set Up Cross-Region Replication Rule: Configure replication rules in the source bucket.

1. Click into your **Source bucket** (my-s3techie) and go to the **Management** tab.
2. Scroll down to **Replication rules** and click **Create replication rule**.
3. Give the rule a name (e.g., ToWestRegionReplication).
4. Under **Source bucket scope**, choose **Apply to all objects in the bucket** (or target a specific folder prefix).
5. Under **Destination**, select **Choose a bucket in this account** and browse/select your destination bucket (my-s3techie-replica-west).
6. Under **IAM role**, select **Create new role** to automatically grant S3 permissions to replicate objects.

7. Click **Save**, and choose **Yes** when prompted to replicate existing objects if needed.

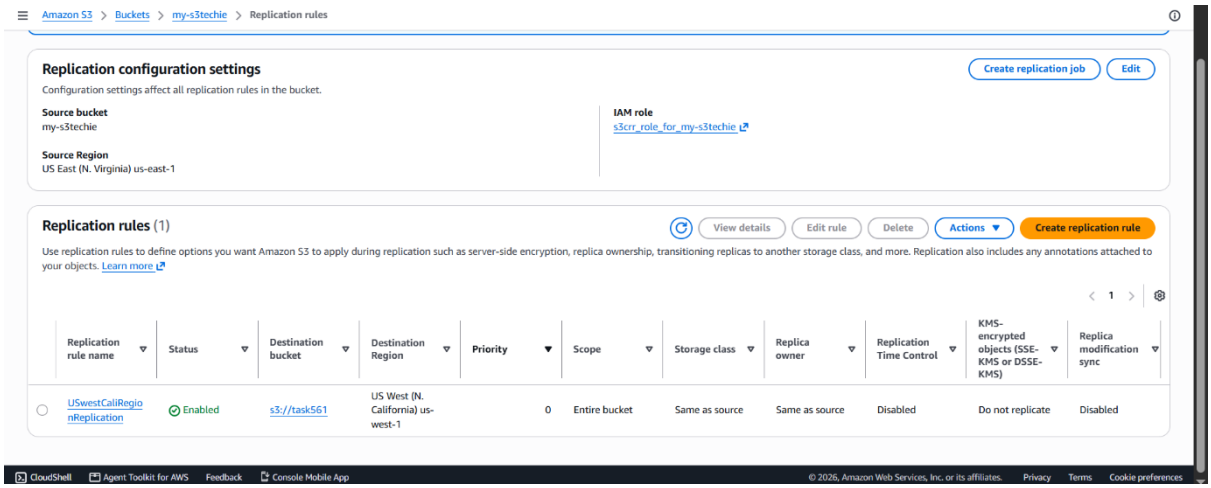


Verification: The Replication rules table lists your newly created rule with a status of "Enabled".

4.Verify Replication Functionality: Test replication with a new upload.

1. Upload a new test file (e.g., replicate-test.txt) to your **Source bucket**.
2. Navigate to your **Destination bucket** in the other region (us-west-2).

Verification: Within a few moments, refresh the Destination bucket objects view—the test file should automatically appear.



4)Configure a bucket policy so only the Admin user can see the objects of the S3 bucket.

Follow these steps to apply a clean, valid bucket policy that grants access exclusively to your admin IAM user/role:

1.Open Bucket Policy Editor: Navigate to bucket permissions.

1. Open the Amazon S3 console and click on your bucket (my-s3techie).

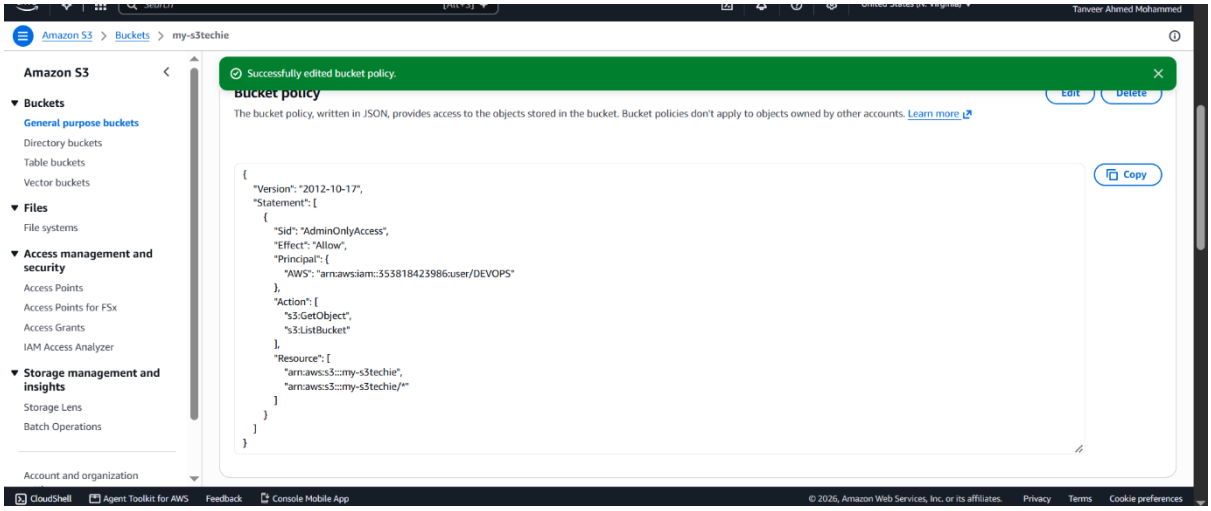
2. Go to the **Permissions** tab.
3. Scroll down to **Bucket policy** and click **Edit**.

Verification: The JSON policy editor text box opens.

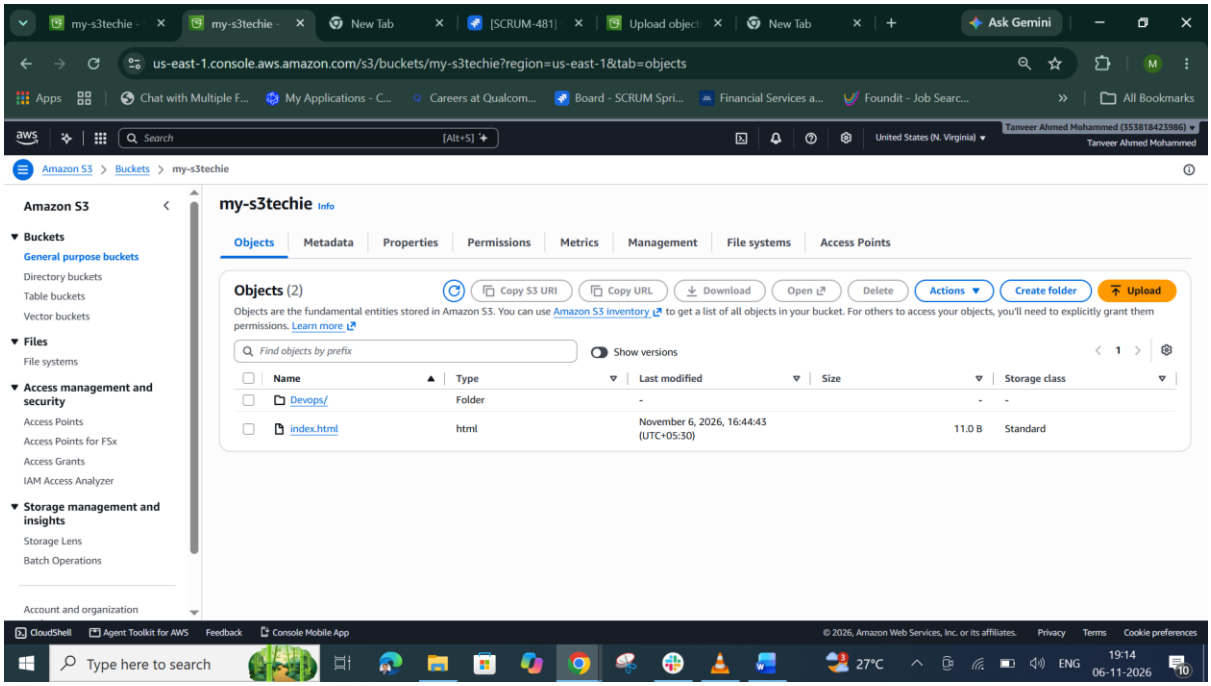
2.Paste and update the JSON Policy: Apply the admin-only policy.

1. Clear any existing policy and paste the following JSON template:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AdminOnlyAccess",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam:: "
      },
      "Action": [
        "s3:GetObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::my-s3techie",
        "arn:aws:s3:::my-s3techie/*"
      ]
    }
  ]
}
```



Verification: The policy saves successfully without displaying any red API error banners, and the policy summary shows the updated statement.



5)Set up lifecycle policies to automatically transition or delete objects based on specific criteria.

Overview of Lifecycle Rules

Lifecycle configuration rules allow you to define actions for Amazon S3 to take on objects during their lifecycle, helping you optimize storage costs:

- **Transition Actions:** Automatically move objects to cheaper storage tiers (e.g., Standard-IA or Glacier) as they age.
- **Expiration Actions:** Permanently delete objects (such as temporary logs or old backups) after a defined period.

Step-by-Step Implementation Guide

1.Navigate to Lifecycle Rules: Open bucket management.

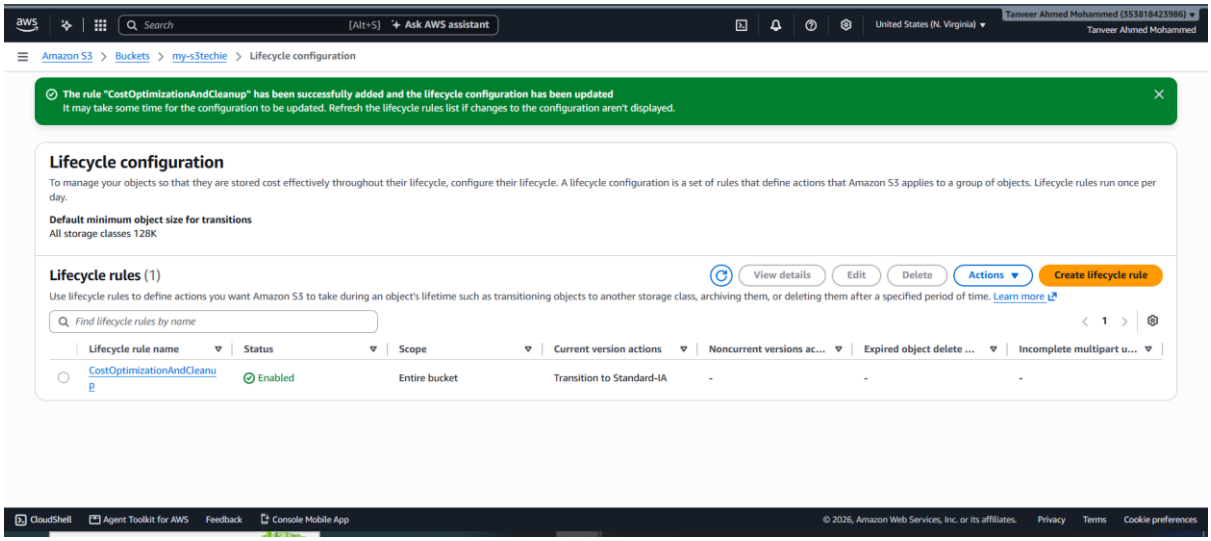
1. Open the **Amazon S3 console** and click on your bucket (my-s3techie).
2. Go to the **Management** tab.
3. Scroll down to **Lifecycle rules** and click **Create lifecycle rule**.

2.Configure Rule Settings: Define rule name and scope.

1. Enter a descriptive **Lifecycle rule name** (CostOptimizationAndCleanup).
2. Under **Rule scope**, select **Apply to all objects in the bucket** (or select a specific prefix like logs/ if you only want the rule to target specific folders/files).

3.Set Up Transitions: Configure storage class transitions.

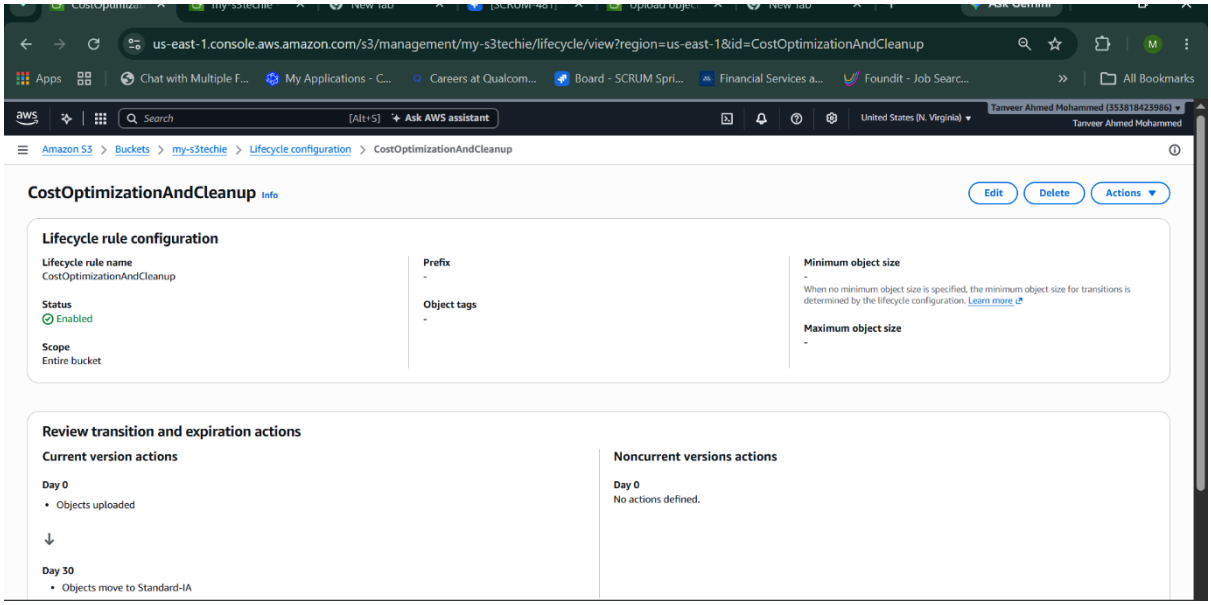
1. Check **Transition current versions of objects between storage classes**.
2. Configure your transition timeline:
    - **Standard-Infrequent Access (Standard-IA):** Set to transition **30 days** after object creation.
    - **Glacier Flexible Archive:** Set to transition **90 days** after object creation.



4.Set Up Expiration: Configure data cleanup / expiration.

1. Check **Expire current versions of objects**.
2. Set the expiration timeframe (e.g., permanently delete objects **365 days** after creation).
3. Review the summary panel at the bottom and click **Create rule**.

Verification: Your new lifecycle rule appears under the Lifecycle rules table with an **Enabled** status.



6)Push some objects to S3 using the AWS CLI

. Upload a Single File (aws s3 cp)

To upload a single local file to your S3 bucket:

Create a local directory and add a test file

mkdir local-folder

echo "File inside folder" > local-folder/testfile.txt

Now run the sync command again

aws s3 sync ./local-folder/ s3://my-s3techie/folder/

2. Upload/Sync an Entire Directory (aws s3 sync)

If you have multiple files or an entire folder locally that you want to push to S3 in bulk:

Sync all files from a local directory to your S3 bucket

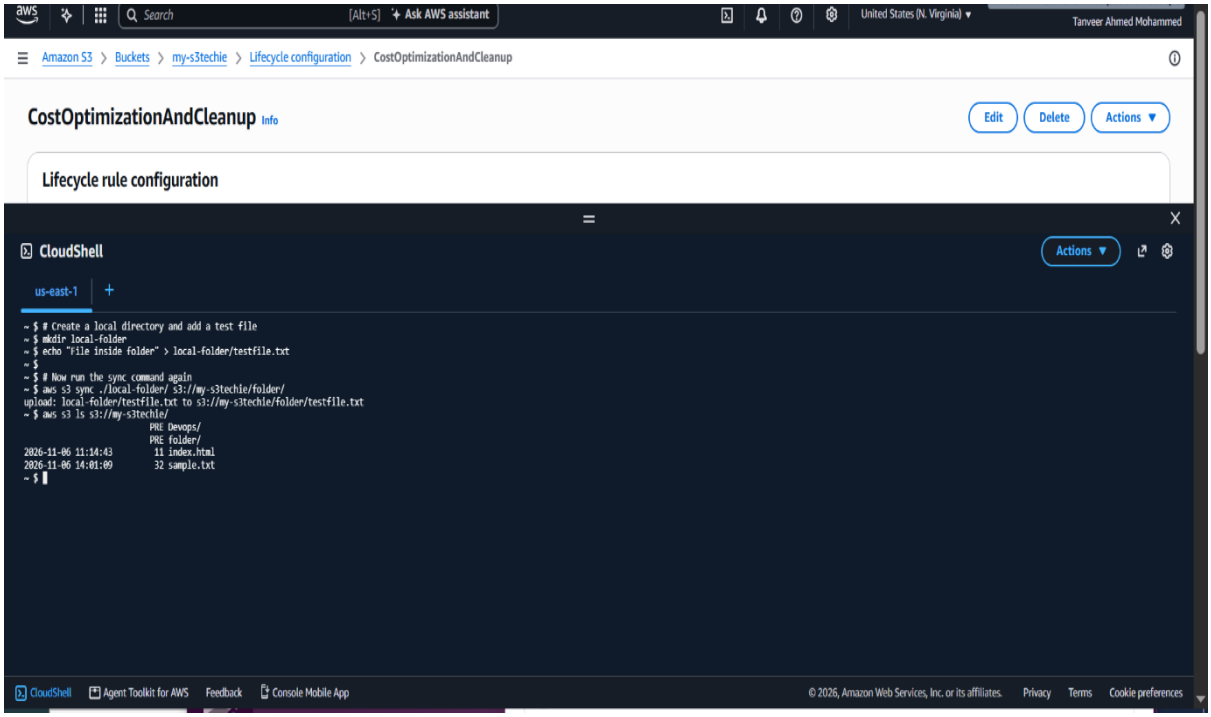
aws s3 sync ./local-folder/ s3://my-s3techie/folder/# Sync all files from a local directory to your S3 bucket

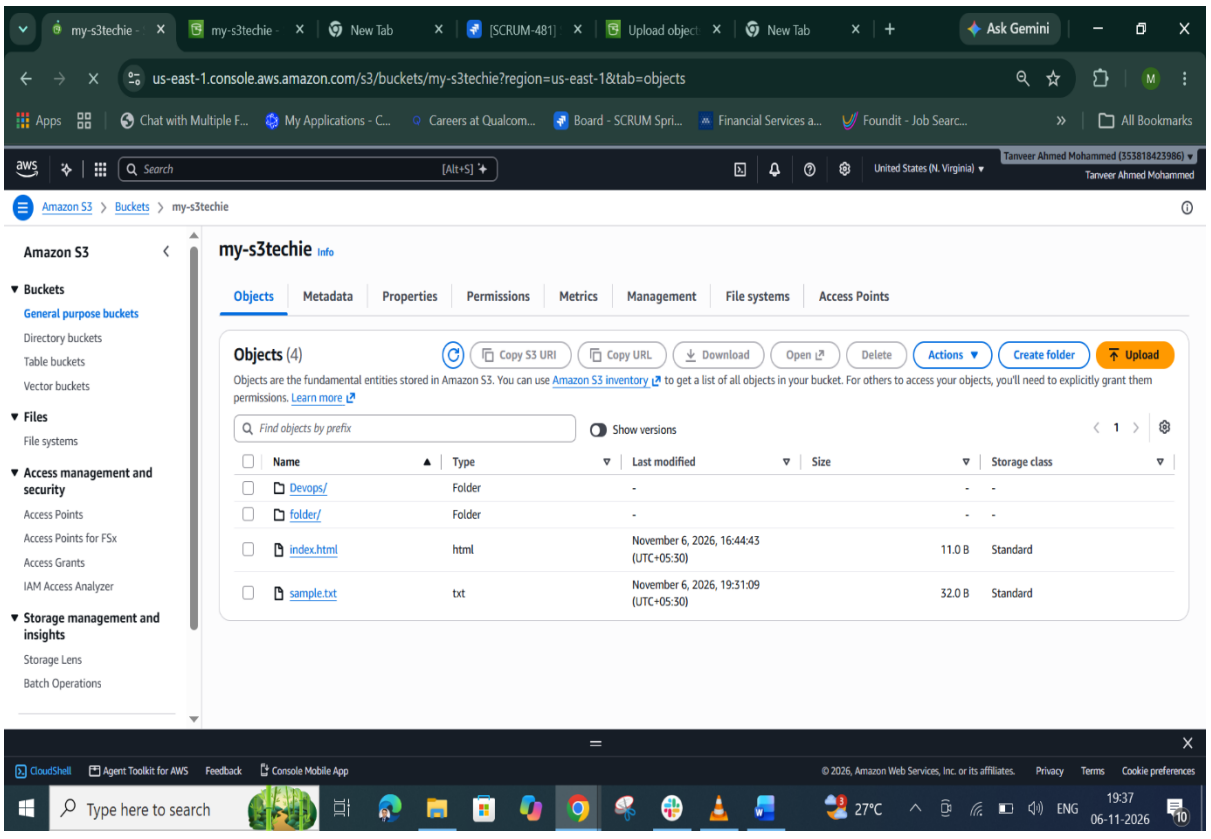
aws s3 sync ./local-folder/ s3://my-s3techie/folder/

3. Verify the Uploaded Objects

You can immediately verify that your objects were successfully pushed by listing the contents of your bucket:

aws s3 ls s3://my-s3techie/





Here, you've successfully pushed objects via CLI.

7) Write a Bash script to create an S3 bucket.

Create the Script File in CloudShell

Run this command in your CloudShell terminal to create and open the script file using nano:

nano create_bucket.sh

```
#!/bin/bash

# Check if bucket name argument is provided

if [ -z "$1" ]; then

  echo "Usage: $0 <bucket-name> <region>"

  exit 1

fi

BUCKET_NAME=$1

REGION=${2:-us-east-1}

echo "Creating S3 bucket: $BUCKET_NAME in region: $REGION..."

if [ "$REGION" == "us-east-1" ]; then

  aws s3api create-bucket --bucket "$BUCKET_NAME"

else

  aws s3api create-bucket --bucket "$BUCKET_NAME" \

    --region "$REGION" \

    --create-bucket-configuration LocationConstraint="$REGION"

fi

if [ $? -eq 0 ]; then

  echo "Successfully created bucket: $BUCKET_NAME"

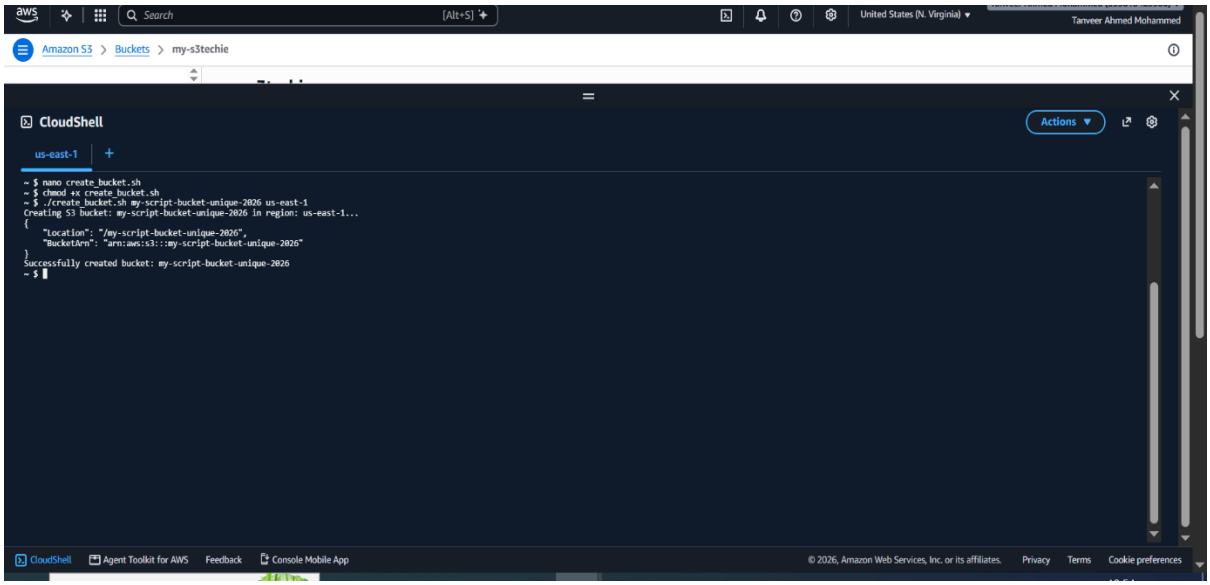
else

  echo "Failed to create bucket."

  exit 1

fi
```

Press Ctrl + O, then hit Enter to save. Press Ctrl + X to exit the editor.



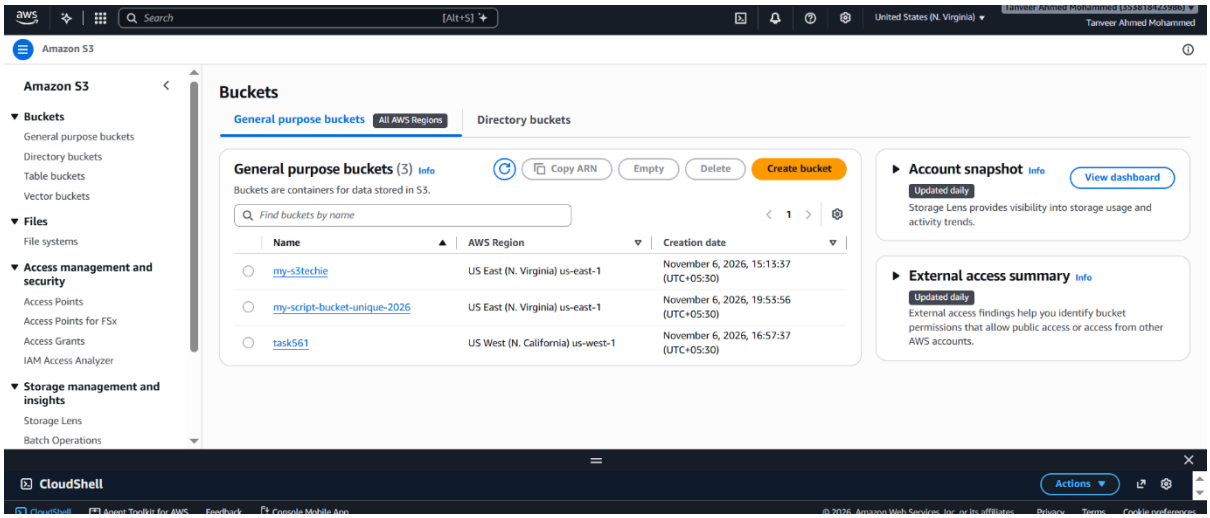
Make the Script Executable and Run It

Make the script executable

chmod +x create_bucket.sh

Run the script with your unique bucket name and region

./create_bucket.sh my-script-bucket-unique-2026 us-east-1



8)Upload a 1 GB file to S3 using the CLI.

Generate a 1 GB Test File Locally

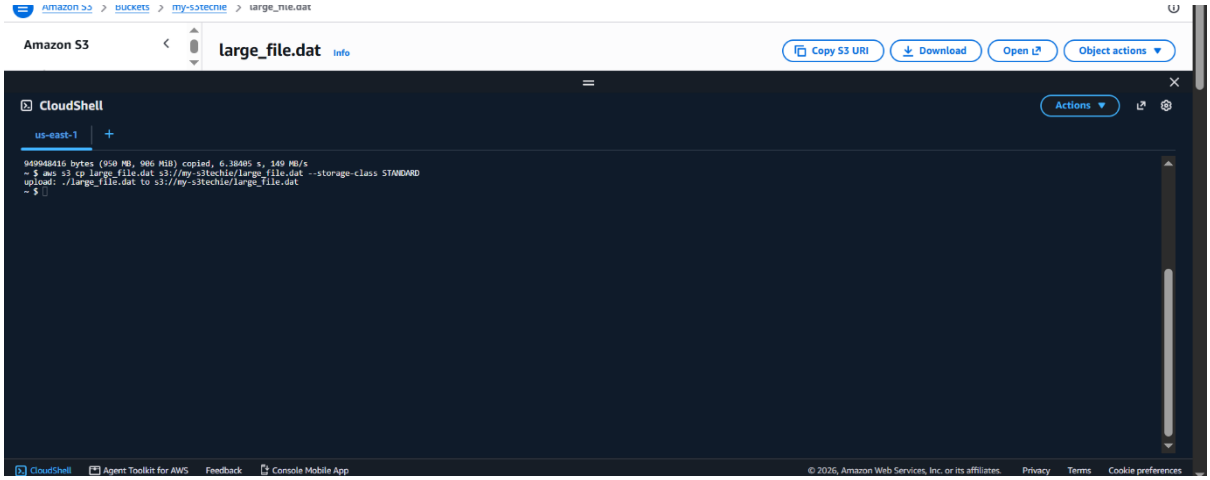
Use the dd command to create a 1 GB file filled with zeros:

dd if=/dev/zero of=large_file.dat bs=1M count=1024

Upload the 1 GB File to Your S3 Bucket

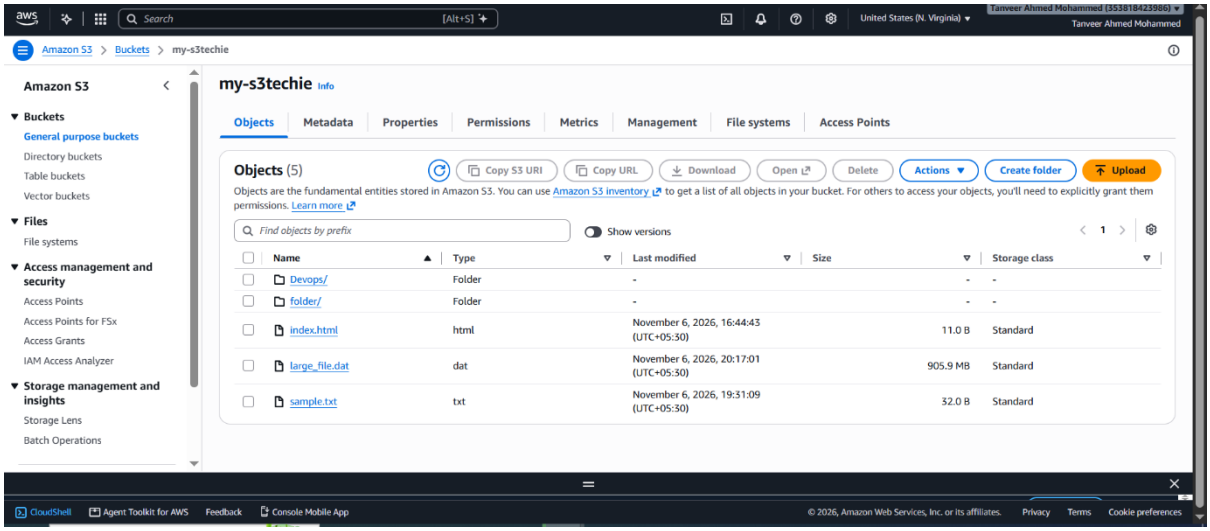
Upload the file using the standard aws s3 cp command. The AWS CLI will automatically handle multipart uploads for large files efficiently:

aws s3 cp large_file.dat s3://my-s3techie/large_file.dat --storage-class STANDARD



Inside that large_file.dat file, **there is actually nothing except empty space (null bytes / zeros).**

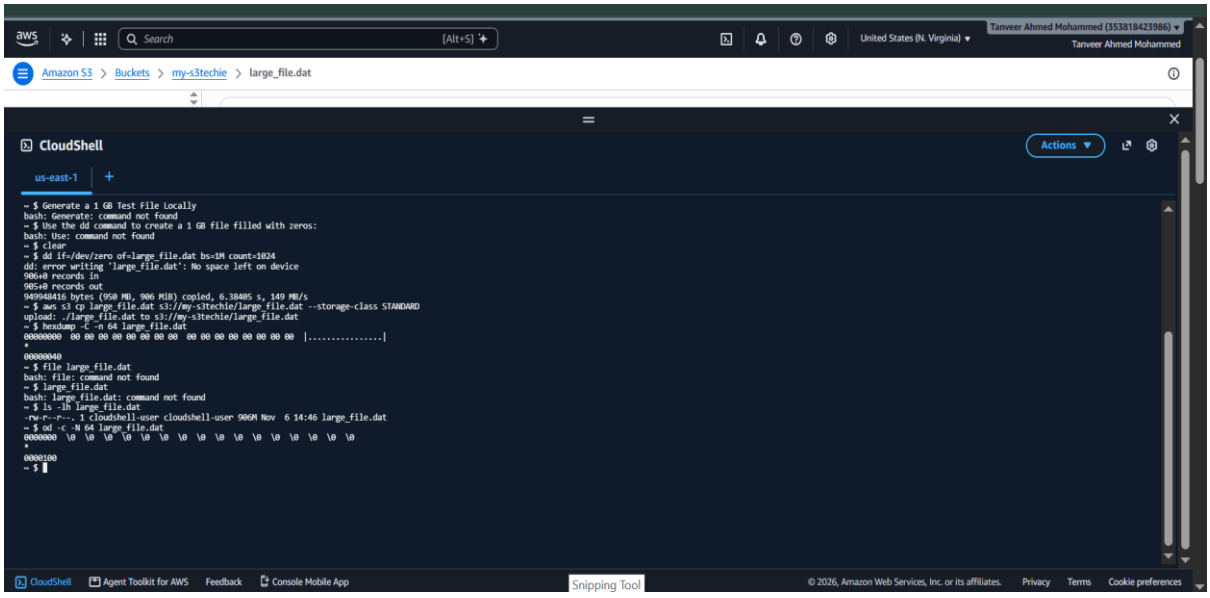
Because the command used if=/dev/zero (the Linux zero device), it filled the entire 1 GB file with a continuous stream of binary zeros (\0).



Inspect the first few bytes using hexdump or od:

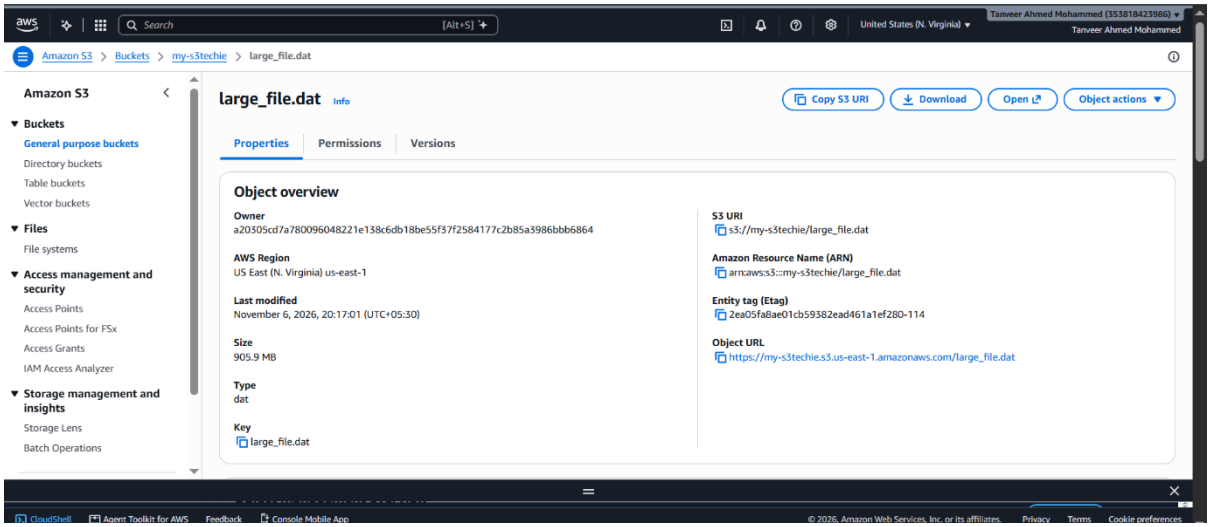
hexdump -C -n 64 large_file.dat

You will see rows of 00000000 00 00 00 00 ... which literally stands for hexadecimal zeros.



To check its size and details: ls -lh large_file.dat

To view the first few lines of hex zeros safely (using od instead of hexdump): od -c -N 64 large_file.dat



Here 1Gb file has created Successfully.

We use a file like this in DevOps and cloud engineering purely as a **dummy test payload** to measure upload speeds, test multipart upload behavior, and verify how S3 handles large file transfers.